EXPRESSÕES REGULARES

Expressões regulares ou "regex" nos permitirão examinar padrões dentro do nosso código. Por exemplo, podemos querer validar se um endereço de e-mail está formatado corretamente. Expressões regulares nos permitirão examinar expressões dessa maneira. Para começar, digite o código validate.py na janela do terminal. Em seguida, codifique conforme abaixo no editor de texto:

```
email = input("What's your email? ").strip()

if "@" in email:
    print("Valid")
else:
    print("Invalid")
```

Observe que strip removerá os espaços em branco no início ou no final da entrada. Ao executar este programa, você verá que, desde que um símbolo @ seja inserido, o programa considerará a entrada válida.

Você pode imaginar, no entanto, que alguém poderia inserir @@ sozinho e a entrada poderia ser considerada válida. Poderíamos considerar um endereço de e-mail como tendo pelo menos um @ e um . em algum lugar dentro dele. Modifique seu código da seguinte forma:

```
email = input("What's your email? ").strip()

if "@" in email and "." in email:
    print("Valid")
else:
    print("Invalid")
```

Observe que, embora isso funcione conforme o esperado, nosso usuário poderia ser adversário, digitando simplesmente @. resultaria no programa retornando válido.

Podemos melhorar a lógica do nosso programa da seguinte forma:

```
email = input("What's your email? ").strip()
```

```
username, domain = email.split("@")

if username and "." in domain:
    print("Valid")
else:
    print("Invalid")
```

Observe como o método strip é usado para determinar se o nome de usuário existe e se . está dentro da variável de domínio. Ao executar este programa, um endereço de e-mail padrão digitado por você poderia ser considerado válido. Digitando apenas malan@harvard, você perceberá que o programa considera essa entrada inválida.

Podemos ser ainda mais precisos, modificando nosso código da seguinte forma:

```
email = input("What's your email? ").strip()

username, domain = email.split("@")

if username and domain.endswith(".edu"):
    print("Valid")
else:
    print("Invalid")
```

Observe como o método endswith verificará se o domínio contém .edu. Ainda assim, no entanto, um usuário malicioso ainda poderia quebrar nosso código. Por exemplo, um usuário poderia digitar malan@.edu e seria considerado válido.

De fato, poderíamos continuar iterando neste código por conta própria. No entanto, acontece que o Python possui uma biblioteca existente chamada re que possui uma série de funções integradas que podem validar entradas de usuário contra padrões. Uma das funções mais versáteis dentro da biblioteca re é a search. A biblioteca search segue a assinatura re.search(pattern, string, flags=0). Seguindo esta assinatura, podemos modificar nosso código da seguinte forma:

```
import re

email = input("What's your email? ").strip()
```

```
if re.search("@", email):
    print("Valid")
else:
    print("Invalid")
```

Observe que isso não aumenta em nada a funcionalidade do nosso programa. Na verdade, é um passo atrás de certa forma.

Podemos aumentar ainda mais a funcionalidade do nosso programa. No entanto, precisamos avançar nosso vocabulário em torno da validação. Acontece que no mundo das expressões regulares existem certos símbolos que nos permitem identificar padrões. Até este ponto, só estivemos verificando pedaços de texto específicos como @. Acontece que muitos símbolos especiais podem ser passados para o compilador com o objetivo de realizar validação. Uma lista não exaustiva desses padrões é a seguinte:

. qualquer caractere exceto uma nova linha

* 0 ou mais repetições

+ 1 ou mais repetições

? 0 ou 1 repetição

{m}m repetições

{m,n} m-n repetições

Implementando isso dentro do nosso código, modifique o seu da seguinte forma:

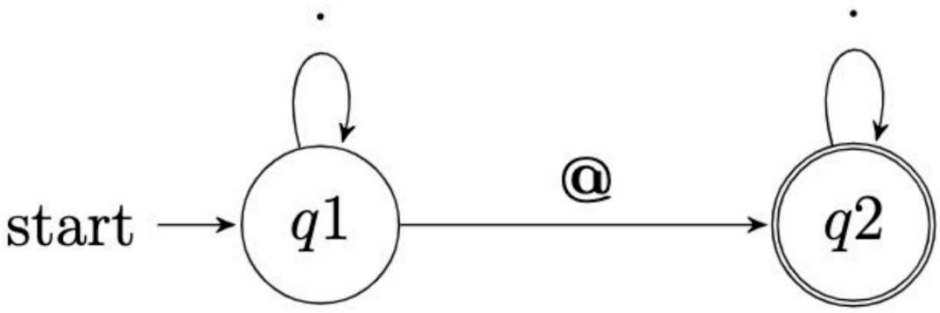
```
import re

email = input("What's your email? ").strip()

if re.search(".+@.+", email):
    print("Valid")
else:
    print("Invalid")
```

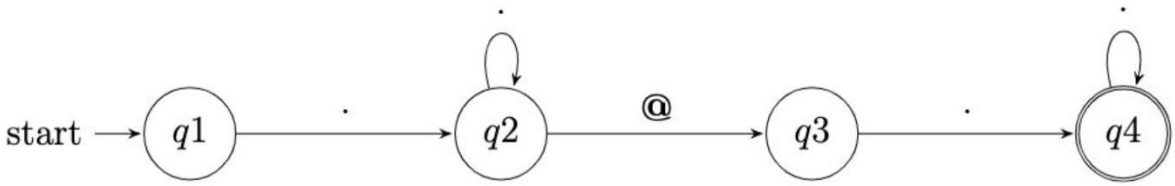
Observe que não nos importamos com qual é o nome de usuário ou domínio. O que nos importa é o padrão. .+ é usado para determinar se há algo à esquerda do endereço de e-mail e se há algo à direita do endereço de e-mail. Executando seu código, digitando malan@, você notará que a entrada é considerada inválida como esperamos.

Se tivéssemos usado uma expressão regular .@. em nosso código acima, você poderia visualizar isso da seguinte maneira: Máquina de Estado.



Observe a representação da máquina de estado da nossa expressão regular. À esquerda, o compilador começa a avaliar a declaração da esquerda para a direita. Uma vez que alcançamos q1 ou questão 1, o compilador lê repetidamente com base na expressão fornecida a ele. Então, o estado é alterado, olhando agora para q2 ou a segunda questão sendo validada. Novamente, a seta indica como a expressão será avaliada repetidamente com base em nossa programação. Então, como representado pelo círculo duplo, o estado final da máquina de estados é alcançado.

Considerando a expressão regular que usamos em nosso código, .+@.+, você pode visualizá-la da seguinte forma: Máquina de Estado.



Observe como q1 é qualquer caractere fornecido pelo usuário, incluindo 'q2' como 1 ou mais repetições de caracteres. Isso é seguido pelo símbolo '@'. Então, q3 procura por qualquer caractere fornecido pelo usuário, incluindo q4 como 1 ou mais repetições de caracteres.

As funções re e re.search e similares procuram padrões. Continuando nossa melhoria deste código, poderíamos melhorar nosso código da seguinte forma:

```
import re

email = input("What's your email? ").strip()
```

```
if re.search(".+@.+.edu", email):
    print("Valid")
else:
    print("Invalid")
```

Observe, no entanto, que alguém poderia digitar malan@harvard?edu e poderia ser considerado válido. Por que isso acontece? Você pode reconhecer que na linguagem de validação, um . significa qualquer caractere!

Podemos modificar nosso código da seguinte forma:

```
import re

email = input("What's your email? ").strip()

if re.search(r".+@.+\.edu", email):
    print("Valid")
else:
    print("Invalid")
```

Observe como utilizamos o "caractere de escape" ou \ como uma forma de considerar o . como parte de nossa string em vez de nossa expressão de validação. Testando seu código, você notará que malan@harvard.edu é considerado válido, onde malan@harvard?edu é inválido.

Agora que estamos usando caracteres de escape, é um bom momento para introduzir "strings brutas". Em Python, strings brutas são strings que não formatam caracteres especiais - em vez disso, cada caractere é considerado ao valor nominal. Imagine \n, por exemplo. Vimos em uma palestra anterior como, em uma string regular, esses dois caracteres se tornam um: um caractere especial de nova linha. Em uma string bruta, no entanto, \n é tratado não como \n, o caractere especial, mas como um único \ e um único n. Colocar um r na frente de uma string indica ao interpretador Python para tratá-la como uma string bruta, semelhante a como colocar um f na frente de uma string indica ao interpretador Python para tratá-la como uma string de formatação:

```
import re
```

```
email = input("What's your email? ").strip()


if re.search(r"^.+@.+\.edu$", email):
    print("Valid")
else:
    print("Invalid")
```

Agora garantimos que o interpretador Python não tratará . como um caractere especial. Em vez disso, simplesmente como um \ seguido de um. - que, em termos de expressão regular, significa corresponder a um "." literal.

Você pode imaginar ainda como nossos usuários podem criar problemas para nós! Por exemplo, você poderia digitar uma frase como Meu endereço de e-mail é malan@harvard.edu. e toda essa frase seria considerada válida. Podemos ser ainda mais precisos em nossa codificação. Acontece que temos mais símbolos especiais à nossa disposição na validação:

^ corresponde ao início da string $ corresponde ao final da string ou imediatamente antes da nova linha no final da string podemos modificar nosso código usando nosso vocabulário adicionado da seguinte forma:

```
import re


email = input("What's your email? ").strip()


if re.search(r"^.+@.+\.edu$", email):
    print("Valid")
else:
    print("Invalid")
```

Observe que isso tem o efeito de procurar esse padrão exato correspondente ao início e ao final da expressão sendo validada. Digitar uma frase como Meu endereço de e-mail é malan@harvard.edu. agora é considerado inválido.

Propomos que podemos fazer ainda melhor! Mesmo que agora estejamos procurando o nome de usuário no início da string, o símbolo @ e o nome do domínio no final, poderíamos digitar quantos símbolos @ desejarmos! malan@@@harvard.edu é considerado válido! Podemos adicionar ao nosso vocabulário da seguinte forma:

[ ] conjunto de caracteres

[^] complementando o conjunto

Usando essas novas habilidades, podemos modificar nossa expressão da seguinte forma:

```
import re

email = input("What's your email? ").strip()

if re.search(r"^[^@]+@[^@]+\.edu$", email):
    print("Valid")
else:
    print("Invalid")
```

Observe que ^ significa corresponder ao início da string. Todo o caminho no final de nossa expressão, $ significa corresponder ao final da string. [^@]+ significa qualquer caractere exceto um @. Então, temos um @ literal. [^@]+.edu significa qualquer caractere exceto um @ seguido de uma expressão terminando em .edu. Digitar malan@@@harvard.edu agora é considerado inválido.

Ainda podemos melhorar ainda mais esta expressão regular. Acontece que existem certos requisitos para o que um endereço de e-mail pode ser! Atualmente, nossa expressão de validação é muito acomodativa. Podemos querer permitir apenas caracteres normalmente usados em uma frase. Podemos modificar nosso código da seguinte forma:

```
import re

email = input("What's your email? ").strip()

if re.search(r"^[a-zA-Z0-9_]+@[a-zA-Z0-9_]+\.edu$", email):
    print("Valid")
else:
    print("Invalid")
```

Observe que [a-zA-Z0-9_] diz à validação que os caracteres devem estar entre a e z, entre A e Z, entre 0 e 9 e potencialmente incluir um símbolo _. Testando a entrada, você verá que muitos erros potenciais do usuário podem ser indicados.

Felizmente, padrões comuns foram incorporados às expressões regulares por programadores dedicados. Neste caso, você pode modificar seu código da seguinte forma:

```
import re

email = input("What's your email? ").strip()

if re.search(r"^\w+@\w+\.edu$", email):
  print("Valid")
else:
  print("Invalid")
```

Observe que \w é o mesmo que [a-zA-Z0-9_]. Obrigado, programadores dedicados!

Aqui estão alguns padrões adicionais que podemos adicionar ao nosso vocabulário:

\d dígito decimal

\D não é um dígito decimal

\s caracteres de espaço em branco

\S não é um caractere de espaço em branco

\w caractere de palavra, além de números e o sublinhado

\W não é um caractere de palavra

Agora, sabemos que não são apenas endereços de e-mail .edu. Podemos modificar nosso código da seguinte forma:

```
import re

email = input("What's your email? ").strip()

if re.search(r"^\w+@\w.+\.(com|edu|gov|net|org)$", email):
```

```
    print("Valid")
else:
    print("Invalid")
```

Observe que | tem o impacto de um ou em nossa expressão.

Adicionando ainda mais símbolos ao nosso vocabulário, aqui estão mais alguns a considerar:

A|B tanto A quanto B

(...) um grupo

(?:...) uma versão não capturadora

SENSIBILIDADE A MAIÚSCULAS E MINÚSCULAS

Para ilustrar como você pode lidar com problemas de sensibilidade a maiúsculas e minúsculas, onde há uma diferença entre EDU e edu e coisas do tipo, vamos retroceder nosso código para o seguinte:

```
import re

email = input("What's your email? ").strip()

if re.search(r"^\w+@\w+\.edu$", email):
    print("Valid")
else:
    print("Invalid")
```

Observe como removemos as instruções | fornecidas anteriormente.

Lembre-se de que dentro da função re.search, há um parâmetro para flags. Algumas variáveis de sinalizador embutidas são:

re.IGNORECASE

re.MULTILINE

re.DOTALL

Considere como você pode usar esses em seu código.

Portanto, podemos alterar nosso código da seguinte forma:

```
import re

email = input("What's your email? ").strip()

if re.search(r"^\w+@\w+\.edu$", email, re.IGNORECASE):
    print("Valid")
else:
    print("Invalid")
```

Observe como adicionamos um terceiro parâmetro re.IGNORECASE. Executando este programa com MALAN@HARVARD.EDU, a entrada agora é considerada válida.

Considere o seguinte endereço de e-mail malan@cs50.harvard.edu. Usando nosso código acima, isso seria considerado inválido. Por que isso pode ser? Como há um ponto adicional, o programa considera isso inválido. Acontece que podemos, olhando nosso vocabulário anteriormente, podemos agrupar ideias.

A|B tanto A quanto B

(...) um grupo

(?:...) uma versão não capturadora

Podemos modificar nosso código da seguinte forma:

```
import re

email = input("What's your email? ").strip()

if re.search(r"^\w+@(\w+\.)?\w+\.edu$", email, re.IGNORECASE):
    print("Valid")
else:
    print("Invalid")
```

Observe como (\w+.)? comunica ao compilador que esta nova expressão pode estar lá uma vez ou não. Portanto, tanto malan@cs50.harvard.edu quanto malan@harvard.edu são considerados válidos.

Curiosamente, as edições que fizemos até agora em nosso código não abrangem completamente todas as verificações que poderiam ser feitas para garantir um endereço de e-mail válido. Na verdade, aqui está a expressão completa que alguém teria que digitar para garantir que um e-mail válido seja inserido:

^[a-zA-Z0-9.!#$%&'*+\/=?^_`{|}~-]+@[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?(?:\.[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?)*$

Existem outras funções dentro da biblioteca re que você pode achar úteis. re.match e re.fullmatch são algumas que você pode achar extremamente úteis.

LIMPANDO A ENTRADA DO USUÁRIO

Você nunca deve esperar que seus usuários sempre sigam suas esperanças por uma entrada limpa. Na verdade, os usuários muitas vezes violarão suas intenções como programador. Há maneiras de limpar seus dados. Na janela do terminal, digite o código format.py. Em seguida, no editor de texto, codifique o seguinte:

```
name = input("What's your name? ").strip()

print(f"hello, {name}")
```

Observe que criamos, essencialmente, um programa "hello world". Executando este programa e digitando David, funciona bem! No entanto, ao digitar Malan, David, observe como o programa não funciona como esperado. Como poderíamos modificar nosso programa para limpar esta entrada?

Modifique seu código da seguinte forma.

```
name = input("What's your name? ").strip()

if "," in name:

   last, first = name.split(", ")

   name = f"{first} {last}"

print(f"hello, {name}")
```

Observe como last, first = name.split(", ") é executado se houver uma , no nome. Em seguida, o nome é padronizado como primeiro e último. Executando nosso código, digitando Malan, David, você pode ver como este programa limpa pelo menos um cenário em que um usuário digita algo inesperado.

Você pode notar que ao digitar Malan,David sem espaço causa um erro no compilador. Como agora conhecemos a sintaxe de expressões regulares, vamos aplicá-la ao nosso código:

```
import re

name = input("What's your name? ").strip()
matches = re.search(r"^(.+), (.+)$", name)
if matches:
    last, first = matches.groups()
    name = first + " " + last
print(f"hello, {name}")
```

Observe que re.search pode retornar um conjunto de correspondências extraídas da entrada do usuário. Se houver correspondências retornadas por re.search. Executando este programa, digitando David Malan, observe como a condição if não é executada e o nome é retornado. Se você executar o programa digitando Malan, David, o nome também será retornado corretamente.

Acontece que podemos solicitar grupos específicos de volta usando matches.group. Podemos modificar nosso código da seguinte forma:

```
import re

name = input("What's your name? ").strip()
matches = re.search(r"^(.+), (.+)$", name)
if matches:
    name = matches.group(2) + " " + matches.group(1)
print(f"hello, {name}")
```

Observe como, nesta implementação, group não é plural (não há s).

Nosso código pode ser ainda mais apertado da seguinte forma:

```
import re

name = input("What's your name? ").strip()
```

```
matches = re.search(r"^(.+), (.+)$", name)
if matches:
    name = matches.group(2) + " " + matches.group(1)
print(f"hello, {name}")
```

Observe como o group(2) e o group(1) são concatenados com um espaço. O primeiro grupo é o que está à esquerda da vírgula. O segundo grupo é o que está à direita da vírgula.

Reconheça ainda que digitar Malan,David sem espaço ainda quebrará nosso código. Portanto, podemos fazer a seguinte modificação:

```
import re

name = input("What's your name? ").strip()
matches = re.search(r"^(.+), *(.+)$", name)
if matches:
    name = matches.group(2) + " " + matches.group(1)
print(f"hello, {name}")
```

Observe a adição do * em nossa declaração de validação. Este código agora aceitará e processará corretamente Malan,David. Além disso, lidará corretamente com David,Malan com muitos espaços antes de David.

É muito comum utilizar re.search como fizemos nos exemplos anteriores, onde matches está em uma linha de código depois. No entanto, podemos combinar essas declarações:

```
import re

name = input("What's your name? ").strip()
if matches := re.search(r"^(.+), *(.+)$", name):
    name = matches.group(2) + " " + matches.group(1)
print(f"hello, {name}")
```

Observe como combinamos duas linhas de nosso código. O operador walrus := atribui um valor da direita para a esquerda e nos permite fazer uma pergunta booleana ao mesmo tempo. Vire a cabeça para o lado e você verá por que isso é chamado de operador walrus.

EXTRAINDO DA ENTRADA DO USUÁRIO

Até agora, validamos a entrada do usuário e limpamos a entrada do usuário. Agora, vamos extrair algumas informações específicas da entrada do usuário. Na janela do terminal, digite o código twitter.py e o seguinte código no editor de texto:

```
url = input("URL: ").strip()

print(url)
```

Observe que se digitarmos https://twitter.com/davidjmalan, ele mostra exatamente o que o usuário digitou. No entanto, como poderíamos extrair apenas o nome de usuário e ignorar o restante da URL?

Você pode imaginar como simplesmente poderíamos eliminar o início da URL padrão do Twitter. Podemos tentar isso da seguinte forma:

```
url = input("URL: ").strip()

username = url.replace("https://twitter.com/", "")

print(f"Username: {username}")
```

Observe como o método replace nos permite encontrar um item e substituí-lo por outro. Neste caso, estamos encontrando parte da URL e substituindo-a por nada. Ao digitar a URL completa https://twitter.com/davidjmalan, o programa efetivamente produz o nome de usuário. No entanto, quais são algumas limitações deste programa atual?

E se o usuário simplesmente digitasse twitter.com em vez de incluir o https:// e coisas do tipo? Você pode imaginar muitos cenários em que o usuário pode inserir ou negligenciar partes da URL que criariam uma saída estranha neste programa. Para melhorar este programa, podemos codificar da seguinte forma:

```
url = input("URL: ").strip()

username = url.removeprefix("https://twitter.com/")

print(f"Username: {username}")
```

Observe como utilizamos o método removeprefix. Este método removerá o início de uma string.

Expressões regulares simplesmente nos permitem expressar sucintamente os padrões e objetivos. Dentro da biblioteca re, há um método chamado sub. Este método nos permite substituir um padrão por outra coisa. A assinatura do método sub é a seguinte:

```
re.sub(pattern, repl, string, count=0, flags=0)
```

Observe como padrão se refere à expressão regular que estamos procurando. Em seguida, há uma string substituta pela qual podemos substituir o padrão. Finalmente, há a string na qual queremos fazer a substituição.

Implementando este método em nosso código, podemos modificar nosso programa da seguinte forma:

```
import re

url = input("URL: ").strip()

username = re.sub(r"https://twitter.com/", "", url)
print(f"Username: {username}")
```

Observe como a execução deste programa e a inserção de https://twitter.com/davidjmalan produzem o resultado correto. No entanto, ainda há alguns problemas presentes em nosso código.

O protocolo, o subdomínio e a possibilidade de que o usuário tenha inserido qualquer parte da URL após o nome de usuário são todos motivos pelos quais este código ainda não é ideal. Podemos abordar ainda mais essas limitações da seguinte forma:

```
import re

url = input("URL: ").strip()

username re.sub(r"^(https?://)?(www\.)?twitter\.com/", "", url)
print(f"Username: {username}")
```

Observe como o ^ foi adicionado à url. Observe também como o . poderia ser interpretado incorretamente pelo compilador. Portanto, escapamos dele usando um \ para torná-lo . Para o propósito de tolerar tanto

http quanto https, adicionamos um ? ao final de https?, tornando o s opcional. Além disso, para acomodar o www, adicionamos (www.)? ao nosso código. Finalmente, caso o usuário decida deixar de fora o protocolo completamente, o http:// ou https:// é tornado opcional usando (https?://).

Ainda assim, estamos esperando cegamente que o usuário tenha inserido uma URL que, de fato, tenha um nome de usuário. Usando nosso conhecimento de re.search, podemos melhorar ainda mais nosso código.

```
import re

url = input("URL: ").strip()

matches = re.search(r"^https?://(www\.)?twitter\.com/(.+)$", url, re.IGNORECASE)
if matches:
  print(f"Username:", matches.group(2))
```

Observe como estamos procurando pela expressão regular acima na string fornecida pelo usuário. Em particular, estamos capturando aquilo que aparece no final da URL usando a expressão regular (.+)$. Portanto, se o usuário deixar de inserir uma URL sem um nome de usuário, nenhuma entrada será apresentada.

Ainda mais apertando nosso programa, podemos utilizar nosso operador := da seguinte forma:

```
import re

url = input("URL: ").strip()

if matches := re.search(r"^https?://(?:www\.)?twitter\.com/(.+)$", url, re.IGNORECASE):
  print(f"Username:", matches.group(1))
```

Observe que o ?: informa ao compilador que não é necessário capturar o que está naquele local em nossa expressão regular.

Ainda assim, podemos ser mais explícitos para garantir que o nome de usuário inserido esteja correto. Usando a documentação do Twitter, podemos adicionar o seguinte à nossa expressão regular:

```python
import re

url = input("URL: ").strip()

if matches := re.search(r"^https?://(?:www\.)?twitter\.com/([a-z0-9_]+)", url, re.IGNORECASE):
    print(f"Username:", matches.group(1))
```

Observe que [a-z0-9_]+ informa ao compilador para esperar apenas a-z, 0-9 e _ como parte da expressão regular. O + indica que estamos esperando um ou mais caracteres.

RESUMINDO

Agora, você aprendeu uma nova linguagem de expressões regulares que pode ser utilizada para validar, limpar e extrair a entrada do usuário.

- Expressões Regulares
- Sensibilidade a Maiúsculas e Minúsculas
- Limpeza da Entrada do Usuário
- Extração da Entrada do Usuário